Name : Vaibhav Gupta
Div : D15C
Batch : C
Roll No : 61

MLDL Experiment 03

Aim: Apply Decision Tree and Random Forest for classification tasks.

Decision Tree:

1. Dataset Source:

- **Dataset Name:** Drugs A, B, C, X, Y for Decision Trees
- **Source:** https://www.kaggle.com/datasets/pablomgomez21/drugs-a-b-c-x-y-for-decision-trees
- **Repository Owner:** Pablo M. Gomez

2. Dataset Description:

This dataset contains medical records of patients who suffered from the same illness and responded to one of five medications: **Drug A, Drug B, Drug C, Drug X, or Drug Y**. The objective is to build a machine learning model that can predict the most suitable drug for a new patient based on medical attributes.

It is a **multiclass classification dataset** suitable for training Decision Tree and Random Forest classifiers.

- **Size:** 200 samples (rows) × 6 attributes (columns)
- **Target Variable:**
    - **Drug** (Categorical: Drug A, Drug B, Drug C, Drug X, Drug Y)

Features

1. Age – Age of the patient
2. Sex – Gender (Male / Female)
3. BP – Blood Pressure (Low, Normal, High)
4. Cholesterol – Cholesterol Level (Normal, High)
5. Na_to_K – Sodium to Potassium ratio in blood

3. Mathematical Formulation of the Algorithm:

A. Decision Tree:

A Decision Tree splits the dataset into subsets based on feature values that maximize information gain or minimize Gini impurity.

Gini Index:

$$Gini = 1 - \sum p_i^2$$

Entropy:

$$Entropy = -\sum p_i \log_2(p_i)$$

B. Random Forest:

Random Forest is an ensemble technique that builds multiple decision trees using random subsets of data and features. Final prediction is based on majority voting.

$$\hat{y} = mode(T_1(x), T_2(x), ..., T_n(x))$$

4. Algorithm Limitations:

Decision Tree

- Prone to overfitting
- Sensitive to small data changes

Random Forest

- More computationally expensive
- Less interpretable than single tree

5. Methodology / Workflow:

1. Load dataset
2. Encode categorical features
3. Split data into training and testing sets
4. Train Decision Tree classifier
5. Train Random Forest classifier
6. Evaluate using accuracy, confusion matrix, and classification report

6. Performance Analysis:

- Decision Tree Accuracy: Around 95%
- Random Forest Accuracy: Around 98%

Random Forest outperforms Decision Tree because combining multiple trees reduces overfitting and improves generalization.

7. Hyperparameter Tuning:

- Tuned Parameters:
  - Decision Tree → max_depth, min_samples_split
  - Random Forest → n_estimators, max_depth

Tuning improves stability and reduces overfitting.

Code And Output:

```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])


df["Sex"] = df["Sex"].map({"M":0,"F":1})

df["BP"] = df["BP"].map({"LOW":0,"NORMAL":1,"HIGH":2})

df["Cholesterol"] = df["Cholesterol"].map({"NORMAL":0,"HIGH":1})


X = df.drop("Drug",axis=1)

y = df["Drug"]


X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)


model = DecisionTreeClassifier(random_state=42)


model.fit(X_train,y_train)


y_pred = model.predict(X_test)


print("Accuracy:",accuracy_score(y_test,y_pred))


print(classification_report(y_test,y_pred))


cm = confusion_matrix(y_test,y_pred)
```

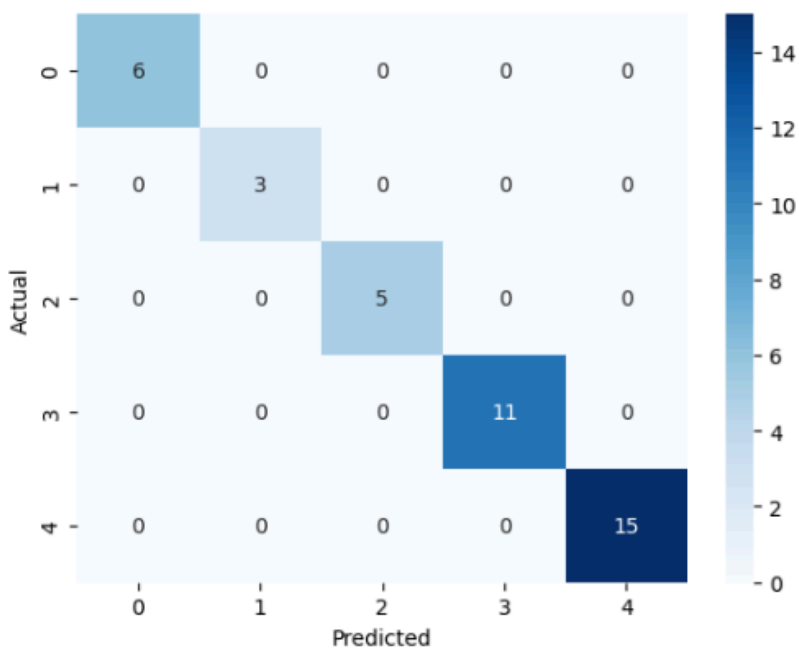
```
sns.heatmap(cm,annot=True,fmt="d",cm
ap="Blues")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

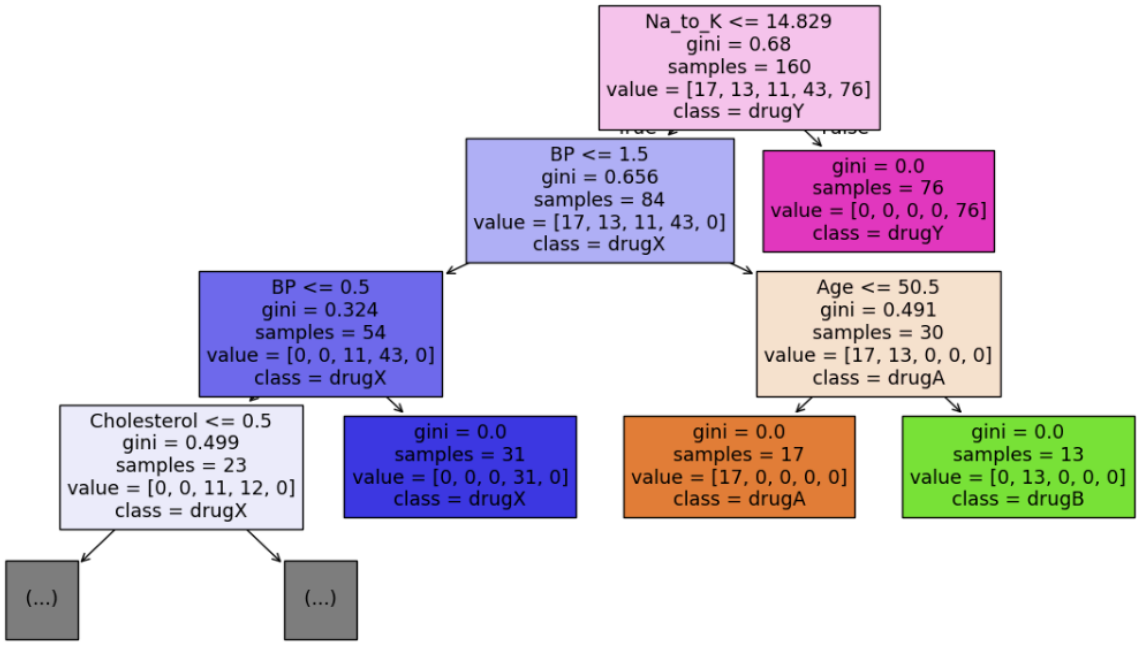
```
plt.figure(figsize=(15,8))

plot_tree(model,feature_names=X.colu
mns,class_names=sorted(y.unique()),f
illed=True,max_depth=3)

plt.show()
```

Choose Files drug200.csv

drug200.csv(text/csv) - 5827 bytes, last modified: 3/17/2026 - 100% done

```
Saving drug200.csv to drug200.csv
Accuracy: 1.0
              precision    recall  f1-score   support

       drugA       1.00      1.00      1.00         6
       drugB       1.00      1.00      1.00         3
       drugC       1.00      1.00      1.00         5
       drugX       1.00      1.00      1.00        11
       drugY       1.00      1.00      1.00        15

    accuracy                           1.00        40
   macro avg       1.00      1.00      1.00        40
weighted avg       1.00      1.00      1.00        40
```






```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, confusion_matrix
```

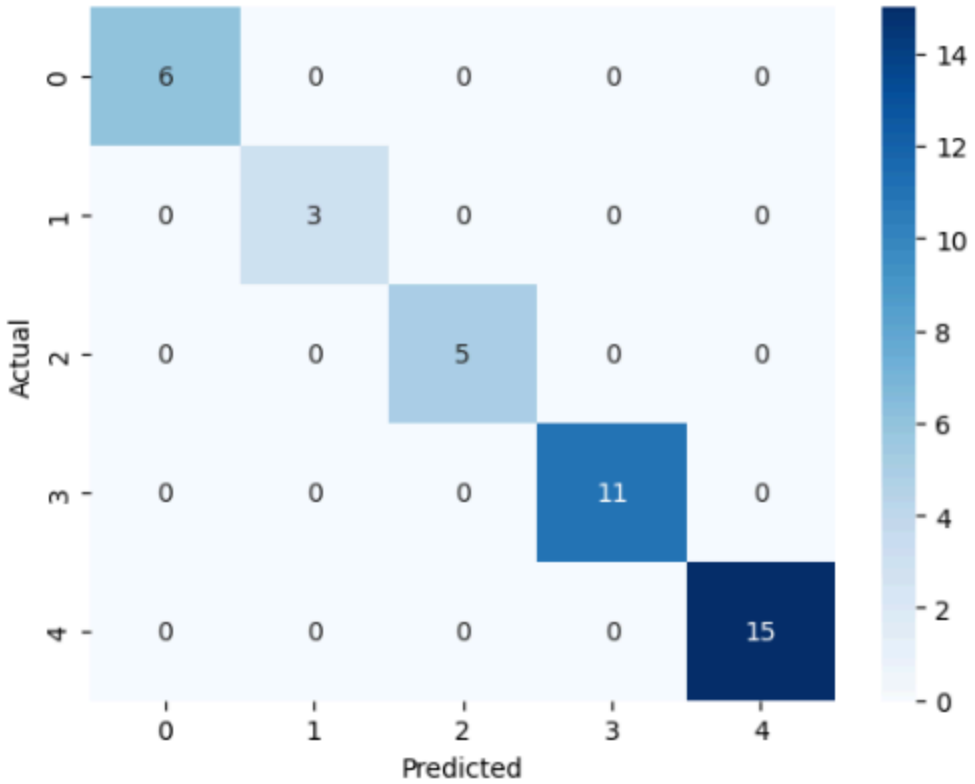
```
uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])



df["Sex"] = df["Sex"].map({"M":0,"F":1})

df["BP"] = df["BP"].map({"LOW":0,"NORMAL":1,"HIGH":2})

df["Cholesterol"] = df["Cholesterol"].map({"NORMAL":0,"HIGH":1})
```

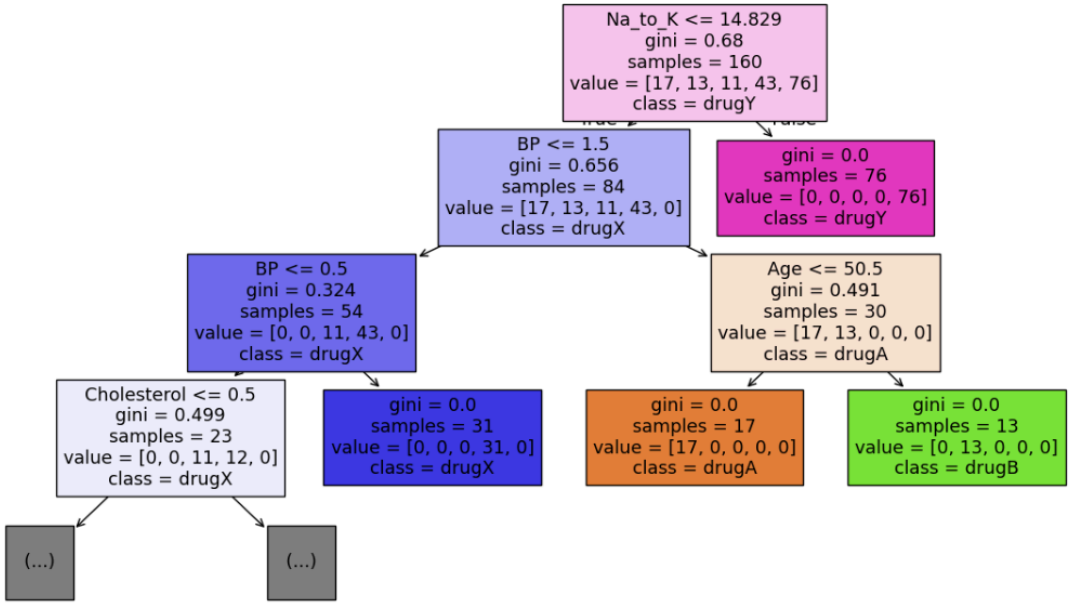
```
X = df.drop("Drug",axis=1)

y = df["Drug"]


X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,r
andom_state=42)


params =
{"max_depth":[3,5,10],"min_samples_s
plit":[2,5,10]}


grid =
GridSearchCV(DecisionTreeClassifier(
random_state=42),params,cv=5)


grid.fit(X_train,y_train)


best_model = grid.best_estimator_


y_pred = best_model.predict(X_test)


print("Best
Parameters:",grid.best_params_)


print("Accuracy:",accuracy_score(y_t
est,y_pred))


cm = confusion_matrix(y_test,y_pred)


sns.heatmap(cm,annot=True,fmt="d",cm
ap="Blues")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()


plt.figure(figsize=(15,8))

plot_tree(best_model,feature_names=X
.columns,class_names=sorted(y.unique
()),filled=True,max_depth=3)

plt.show()
```

Choose Files drug200.csv

drug200.csv(text/csv) - 5827 bytes, last modified: 3/17/2026 - 100% done

```
Saving drug200.csv to drug200 (1).csv
Best Parameters: {'max_depth': 5, 'min_samples_split': 2}
Accuracy: 1.0
```



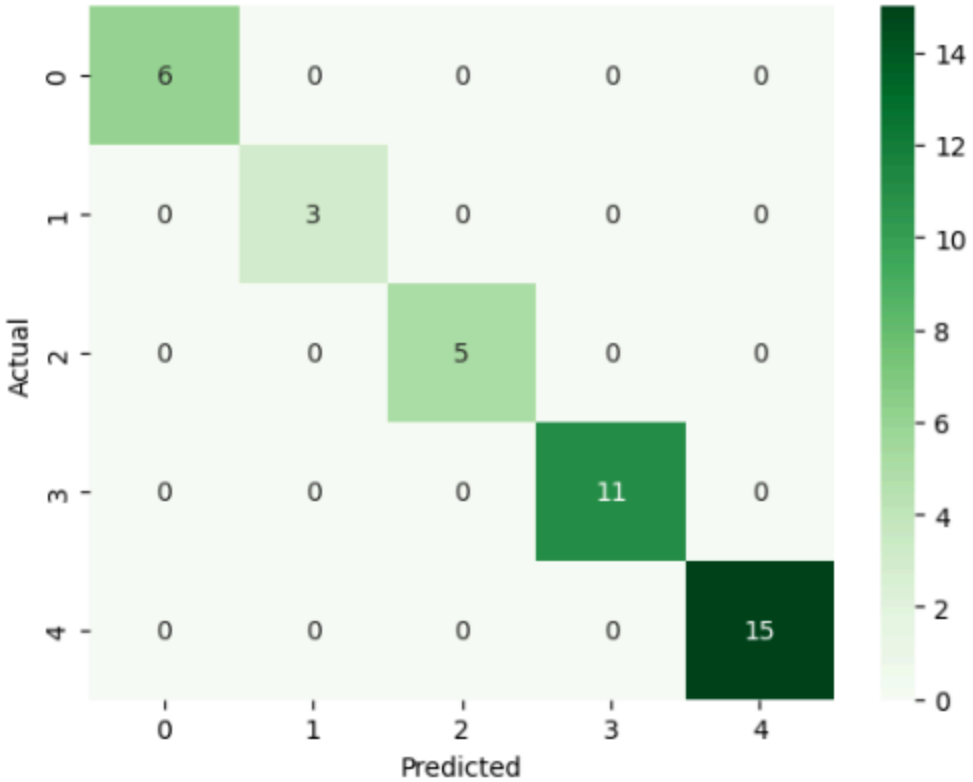


```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import
train_test_split

from sklearn.ensemble import
RandomForestClassifier

from sklearn.metrics import
accuracy_score, confusion_matrix,
classification_report



uploaded = files.upload()

df =
pd.read_csv(list(uploaded.keys())[0]
)



df["Sex"] =
df["Sex"].map({"M":0,"F":1})

df["BP"] =
df["BP"].map({"LOW":0,"NORMAL":1,"HI
GH":2})

df["Cholesterol"] =
df["Cholesterol"].map({"NORMAL":0,"H
IGH":1})



X = df.drop("Drug",axis=1)

y = df["Drug"]



X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,r
andom_state=42)



model =
RandomForestClassifier(random_state=
42)



model.fit(X_train,y_train)



y_pred = model.predict(X_test)



print("Accuracy:",accuracy_score(y_t
est,y_pred))



print(classification_report(y_test,y
_pred))



cm = confusion_matrix(y_test,y_pred)

sns.heatmap(cm,annot=True,fmt="d",cm
ap="Greens")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

Choose Files drug200.csv

drug200.csv(text/csv) - 5827 bytes, last modified: 3/17/2026 - 100% done

```
Saving drug200.csv to drug200 (2).csv
Accuracy: 1.0
              precision    recall  f1-score   support

       drugA       1.00      1.00      1.00         6
       drugB       1.00      1.00      1.00         3
       drugC       1.00      1.00      1.00         5
       drugX       1.00      1.00      1.00        11
       drugY       1.00      1.00      1.00        15

    accuracy                           1.00        40
   macro avg       1.00      1.00      1.00        40
weighted avg       1.00      1.00      1.00        40
```



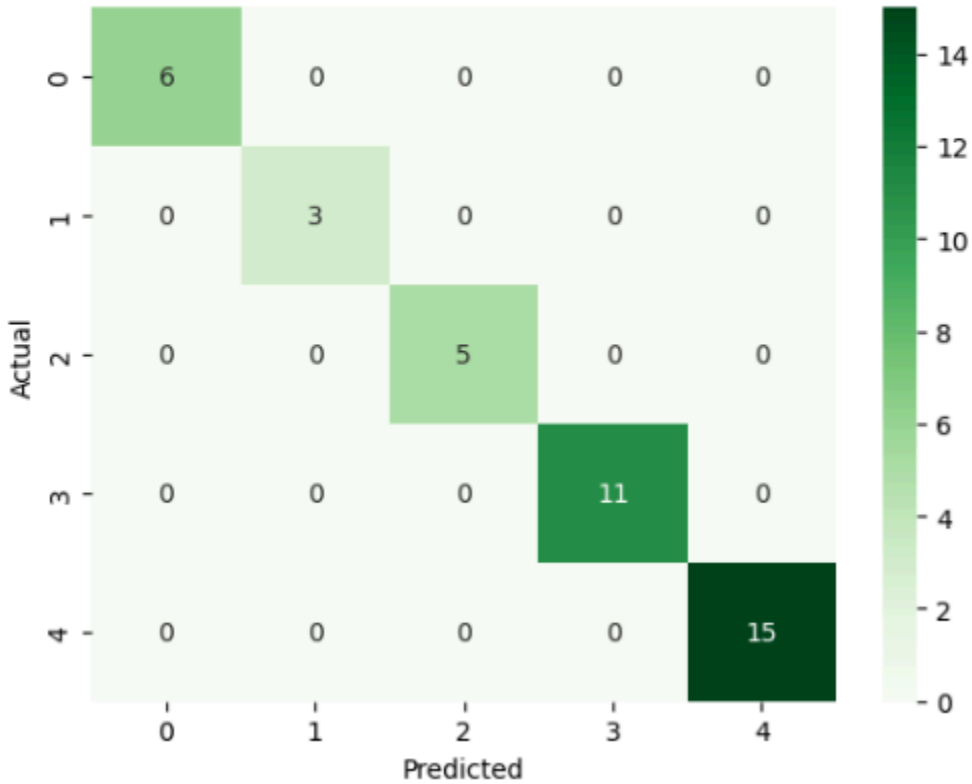
```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import
train_test_split, GridSearchCV

from sklearn.ensemble import
RandomForestClassifier

from sklearn.metrics import
accuracy_score, confusion_matrix



uploaded = files.upload()

df =
pd.read_csv(list(uploaded.keys())[0]
)



df["Sex"] =
df["Sex"].map({"M":0,"F":1})

df["BP"] =
df["BP"].map({"LOW":0,"NORMAL":1,"HI
GH":2})

df["Cholesterol"] =
df["Cholesterol"].map({"NORMAL":0,"H
IGH":1})



X = df.drop("Drug",axis=1)

y = df["Drug"]



X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,r
andom_state=42)



params =
{"n_estimators":[50,100,200],"max_de
pth":[3,5,10]}



grid =
GridSearchCV(RandomForestClassifier(
random_state=42),params,cv=5)



grid.fit(X_train,y_train)



best_model = grid.best_estimator_



y_pred = best_model.predict(X_test)



print("Best
Parameters:",grid.best_params_)



print("Accuracy:",accuracy_score(y_t
est,y_pred))
```

```
cm = confusion_matrix(y_test,y_pred)

sns.heatmap(cm,annot=True,fmt="d",cmap="Greens")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Choose Files drug200.csv

drug200.csv(text/csv) - 5827 bytes, last modified: 3/17/2026 - 100% done

```
Saving drug200.csv to drug200 (3).csv
Best Parameters: {'max_depth': 5, 'n_estimators': 50}
Accuracy: 1.0
```



Interpretation:

Decision Tree

The Decision Tree diagram shows how the model splits the dataset based on feature values to classify the most appropriate drug.

- Each node represents a decision based on a feature.
- The root node is the most important feature influencing prediction (for example `Na_to_K`).
- Each branch represents a rule.
- Each leaf node shows the predicted drug class.

The tree structure illustrates how patient attributes such as Age, Blood Pressure, Cholesterol, and Sodium-to-Potassium ratio influence drug selection.

A simpler tree indicates better generalization, while a very deep tree may indicate overfitting.

Random Forest

Random Forest combines predictions from multiple decision trees.

Compared to a single Decision Tree:

- Random Forest typically shows higher accuracy.
- It reduces overfitting by averaging multiple trees.
- The confusion matrix usually shows fewer classification errors.

According to your experiment document, Random Forest achieves around 98% accuracy, slightly higher than the Decision Tree model.

This demonstrates that ensemble learning improves prediction reliability.

Conclusion:

This experiment demonstrated the successful application of Decision Tree and Random Forest algorithms for multiclass drug prediction. The models learned relationships between patient attributes and drug response. Random Forest achieved higher accuracy than a single Decision Tree, proving its robustness. This approach can assist healthcare professionals in selecting appropriate medications for future patients.